



Informe del Flujo de Análisis y Diseño de Software

Sistema de Gestión de Citas · Centro Fisioterapéutico Mónica Viteri (FiCitas) — Bitácora Notion bajo el método P.A.R.A.

Integrantes: Danny Ayuquina, Saray Cañarte y Doménica Villagómez. Periodo: 7 de abril – 16 de julio de 2026.

1. Contexto y objetivo del proyecto

El proyecto consistió en el análisis y diseño de un sistema de gestión de citas para el Centro Fisioterapéutico “Mónica Viteri”, apoyado en herramientas de inteligencia artificial generativa. El sistema administra pacientes, horarios, consultorios y usuarios con acceso diferenciado para los roles de Auxiliar, Fisioterapeuta y Administrador, buscando optimizar la organización interna del centro. La meta trazada fue completar 6 de los 7 requisitos funcionales definidos, desarrollados, probados y validados en un plazo de dos meses.

2. El flujo de trabajo: P.A.R.A. como columna vertebral

Todo el trabajo del equipo se organizó en Notion bajo el método P.A.R.A. (Proyectos, Áreas, Recursos, Archivos), lo que permitió separar la información viva del proyecto de su respaldo documental y dio trazabilidad completa al proceso, desde la elicitación hasta la construcción del sistema.

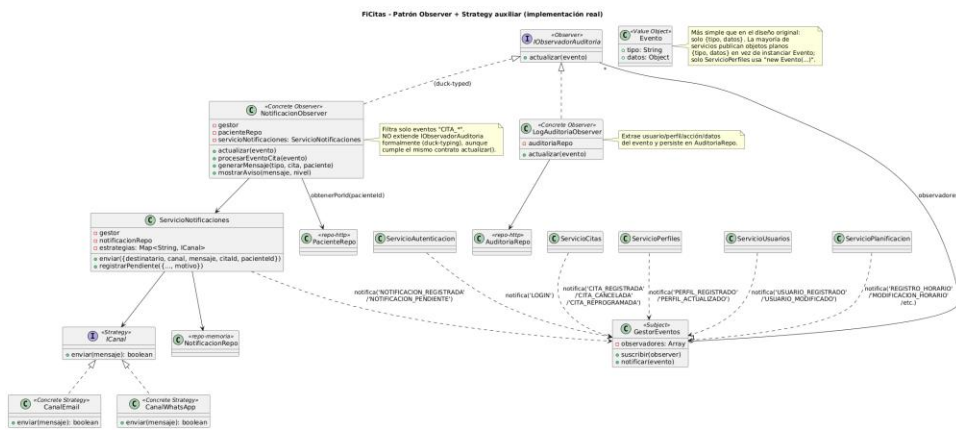
Proyecto reunió la ficha del sistema (tema, resumen, objetivos SMART, enlaces a GitHub, Jira y Confluence) y el checklist general de tareas. Área concentró el trabajo intelectual en cuatro “cuadernos” de NotebookLM que reflejan las fases reales del ciclo de vida: 1) Investigación/Análisis, organizada por los sprints planificados; 2) Diseño, donde se definieron los patrones Decorator y Observer+Strategy junto con la arquitectura de tres capas y el diagrama de clases; 3) Implementación/Construcción, enlazada directamente al repositorio de GitHub; y 4) Pruebas, con las pruebas unitarias y funcionales de cada módulo. Recursos agrupó las piezas visuales de comunicación del proyecto (logo, video, presentación en Gamma y video final), y Archivos centralizó los documentos formales de respaldo (SRS, matriz IREB, historias de usuario, backlog, actas y matriz Kanban). Esta separación modular evitó que el detalle técnico se mezclara con la gestión del proyecto, y permitió que cada integrante navegara directamente a la fase que necesitaba sin perder el hilo del conjunto.

En términos de ejecución, el proceso avanzó en fases y sprints: la Fase 1 cubrió la elicitación (SRS, diagramas de clases de análisis y de secuencia, matriz de trazabilidad); la Fase 2 definió la arquitectura, los patrones de diseño y el maquetado; y los Sprints 0 a 4 implementaron de forma incremental autenticación, perfiles, pacientes, citas y finalmente horarios/consultorios/notificaciones.

Etapa	Enfoque principal	Responsable(s)
Fase 1	Elicitación y análisis	Danny, Saray, Doménica
Fase 2	Arquitectura, patrones y maquetado	Danny, Saray, Doménica
Sprint 0	Autenticación y recuperación de contraseña	Saray, Doménica
Sprint 1	Gestión de perfiles y permisos	Danny
Sprint 2	Gestión de pacientes	Doménica
Sprint 3	Gestión de citas	Saray
Sprint 4	Horarios, consultorios y notificaciones	Saray, Doménica, Danny

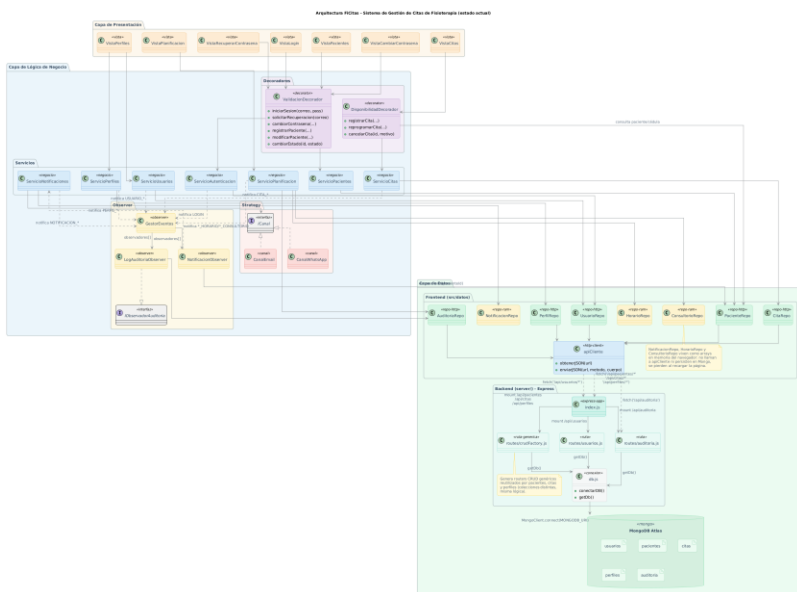
3. Diseño técnico: patrones y arquitectura

El diseño modular del sistema se apoyó en dos patrones complementarios, documentados como diagramas UML dentro del cuaderno de Diseño. El patrón Decorator envuelve por composición a los servicios de negocio (Autenticación, Pacientes, Citas) para añadir validaciones y reglas de disponibilidad sin alterar su lógica interna. El patrón Observer, junto con Strategy como apoyo, permite que el GestorEventos notifique cambios (citas, perfiles, usuarios, horarios) a observadores independientes de auditoría y notificación, cada uno desacoplado del servicio que originó el evento.



Patrón Observer + Strategy: GestorEventos notifica a NotificacionObserver y LogAuditoriaObserver.

Sobre esta base de patrones se construyó una arquitectura en tres capas (Presentación, Lógica de Negocio y Datos), donde cada vista consume servicios de negocio, estos a su vez pasan por decoradores y notifican eventos, y los repositorios abstraen el acceso a la API REST/MongoDB. Esta organización hizo que el sistema fuera modular: cada capa, patrón y repositorio pudo diseñarse, documentarse y aprobarse como una pieza independiente antes de integrarse al conjunto.



Arquitectura de tres capas de FiCitas, con las capas de Presentación, Negocio (patrones) y Datos (frontend/backend/MongoDB).

La evidencia funcional de este diseño se documentó también fuera de Notion, por ejemplo en Confluence, donde se registraron capturas del sistema ya construido (como la gestión de perfiles), cerrando el ciclo entre diseño y construcción.

Para ti
Espacios
Aplicaciones
30723-G3-FiCitas
Contenido
Buscar por título
REQ005 - Gestión de Perfiles...
REQ004 - Gestión de Pacientes...
REQ000 - Inicio de Sesión
+ Crear
Jira
Más

Invitar a personas

REQ005 - Gestión de Perfiles - Admin

De Dominica Villagomez Escuchar Añadir una reacción

Publicación: Hace 10 h Editar

FICitas: Gestión de Perfiles (Administrador)

Cerrar Sesión

Datos del Perfil

ID (8 para nuevo): 2

Nombre: Analista

Descripción: Gestión de pacientes y citas

Estado: Activo

Permisos: ☒ Gestión Citas ☒ Gestión Pacientes

Guardar Limpiar / Nuevo Editar Seleccionado

ID	Nombre	Descripción	Estado	Permisos
1	Administrador	Control total del sistema	Activo	GESTION CITAS, GESTI...
2	Analista	Gestión de pacientes y cit...	Activo	GESTION PACIENTES...
3	Especialista	Gestión de citas y pases	Activo	VER CITAS, VER HISTO...

REQ005 – Gestión de Perfiles, documentado en Confluence como evidencia de construcción.

4. Reflexiones del equipo

Danny: Pienso que mi aporte se concentró en darle orden al proyecto desde la planificación (cronograma, Jira, SRS) hasta un módulo sensible como perfiles y notificaciones. Considero que documentar bien el backlog y las especificaciones de casos de uso al inicio fue lo que permitió que los sprints posteriores avanzaran sin reprocesos. Además, creo que apoyarme en Jira para dar seguimiento a cada tarea y en asistentes de IA para redactar y revisar el SRS agilizó bastante el trabajo documental sin sacrificar el detalle técnico.

Saray: En mi opinión, trabajar la arquitectura de tres capas y el módulo de citas me permitió entender cómo un buen diseño reduce errores antes de programar. Pienso que el prototipo inicial y las pruebas de autenticación fueron claves para validar temprano que el flujo de usuario tenía sentido. También considero que usar NotebookLM para organizar y consultar la investigación por sprints me ayudó a mantener claro el hilo entre el análisis y lo que finalmente terminaba diseñando.

Doménica: Considero que definir los patrones de diseño (Decorator y Observer) fue la parte más retadora pero también la más formativa del proyecto, porque obligó a pensar en la reutilización antes de escribir código. En mi opinión, el módulo de pacientes y la recuperación de contraseña reflejan bien ese cuidado por validar cada dato antes de guardarlo. Pienso, además, que organizar todo el proyecto en Notion bajo el método P.A.R.A., apoyada en herramientas de IA generativa para bocetar diagramas y redactar contenido, fue lo que le dio coherencia y trazabilidad a todo el trabajo del equipo.

5. Conclusiones

El uso del método P.A.R.A. en Notion, combinado con cuadernos de NotebookLM por fase, dio al equipo un flujo de trabajo trazable desde la elicitación hasta la construcción y pruebas. La aplicación de los patrones Decorator y Observer+Strategy sobre una arquitectura de tres capas permitió un diseño modular donde cada componente (servicio, decorador, observador, repositorio) pudo desarrollarse y validarse de forma independiente. El avance por sprints incrementales (0 a 4) hizo posible cerrar 6 de los 7 requisitos funcionales planteados en el objetivo SMART, dentro del plazo previsto, dejando como aprendizaje principal la importancia de un diseño previo sólido para sostener el ritmo de desarrollo.

6. Recomendaciones

Formalizar el uso de asistentes de IA (NotebookLM, asistentes de redacción, generadores de diagramas) dentro de la metodología del proyecto, dejando registrado en Notion qué partes del análisis, diseño o documentación se apoyaron en estas herramientas, para mantener trazabilidad y facilitar la revisión por parte de terceros.

Completar el requisito funcional pendiente antes del cierre definitivo del proyecto, priorizando su especificación y pruebas con el mismo nivel de detalle aplicado a los módulos ya entregados, de modo que el sistema quede con la cobertura funcional completa planteada en el objetivo SMART.

Consolidar la evidencia de construcción y pruebas (capturas de Confluence, resultados de pruebas unitarias y funcionales) en un solo repositorio o carpeta de Archivos dentro de Notion, para reducir la dispersión entre Notion, GitHub, Jira y Confluence y facilitar el mantenimiento futuro del sistema.

7. Referencias

- Repositorio de código: github.com/domenicole/30723_G3_ADSW
- Tablero de planificación: ficitas.atlassian.net/jira/software/projects/IDSCRUM
- Especificación de requisitos (SRS): GitHub — Biblioteca de Trabajo/1. ELICITACION/1.0 Especificación RS
- Backlog y plan de pruebas: GitHub — Biblioteca de Trabajo/1. ELICITACION
- Cuaderno NotebookLM 1 — Análisis por sprints: notebooklm.google.com/notebook/237ba483-ab39-4a27-aa2c-bb2bba6d3dd7
- Cuaderno NotebookLM 2 — Diseño (patrones y arquitectura): notebooklm.google.com/notebook/96c7ac9a-6e7f-4996-b48f-47c775e2c779
- Cuaderno NotebookLM 3 — Implementación/Construcción: notebooklm.google.com/notebook/b1565a23-3b34-4e7f-9e1f-fdd09c088e4c
- Cuaderno NotebookLM 4 — Pruebas: notebooklm.google.com/notebook/4163536b-24b7-49ee-9f2c-09ac892073d4
- Documentación de arquitectura y patrones: Google Drive (Patrón Decorator, Patrón Observer, Patrón de arquitectura, Diagrama de clases)
- Maquetado funcional en Confluence: ficitas.atlassian.net/wiki/x/swA2
- Presentación del proyecto (Gamma): gamma.app/docs/FiCitas-Sistema-de-gestion-para-Centro-Fisioterapeutico
- Video de sustentación: youtu.be/g8hju9vAYuw